

Práctico AYED1 - Sala 2 - 28-10

Práctico 6 - Ejercicio 3a

Elegir E tal que valga la terna:

Var x, y : Nat;

{ True }

x, y := x + 1, **E**

{ y = x + 1 }

¿sale a ojo?

¿E = x + 1? Probemos: si el estado inicial es x -> 3, el estado final será x -> 4, y -> 4, que no cumple la postcondición. Así que **no**.

¿E = x + 2? con x -> 3, terminamos en x -> 4, y -> 5, en este caso sí anda.

Ahora tratemos de obtener **E** haciendo la demostración. La terna vale sí y sólo si:

$$\text{True} \Rightarrow \text{wp.}(x, y := x + 1, \mathbf{E}).(y = x + 1)$$

Supongamos la premisa (True) y veamos la wp:

$$\begin{aligned} & \text{wp.}(x, y := x + 1, E).(y = x + 1) \quad // \text{ la idea acá es llegar a True,} \\ & \quad \quad \quad // \text{ eligiendo en el camino lo que nos convenga que sea } \mathbf{E} \\ \equiv \{ \text{def. :=} \} & \\ & (y = x + 1)(x \leftarrow x + 1, y \leftarrow E) \\ \equiv \{ \text{sust.} \} & \\ & E = (x + 1) + 1 \\ \equiv \{ \text{arit.} \} & \\ & E = x + 2 \\ \equiv \{ \text{elijo } \mathbf{E = x + 2} \} & \\ & x + 2 = x + 2 \\ \equiv \{ \text{logica} \} & \\ & \text{True} \end{aligned}$$

Luego, $E = x + 2$ y el programa queda:

```
Var x, y : Nat;  
{ True }  
x, y := x + 1, x + 2  
{ y = x + 1 }
```

Ejemplo con dos incógnitas

```
Var x, y, z : Nat;  
{ True }  
x, y, z := x + 1, E, F  
{ y = x + 1  $\wedge$  z = 2 * y }
```

Supongamos la precondition (True) y veamos la wp:

wp.(x, y, z := x + 1, E, F).(y = x + 1 $\wedge$ z = 2 * y)

$\equiv$ {def wp para := y sustitución }

E = (x + 1) + 1 $\wedge$ F = 2 * E

$\equiv$ { elijo **E = x + 2** }

x + 2 = x + 2 $\wedge$ F = 2 * (x + 2)

$\equiv$ { elijo **F = 2 * (x + 2)** }

x + 2 = x + 2 $\wedge$ 2 * (x + 2) = 2 * (x + 2)

$\equiv$ { lógica }

True

Ejemplo con dos incógnitas más complicado

```
Var x, y : Nat;  
{ True }  
x, y := x + F, E  
{ y = x + 1 }
```

¿Sale a ojo? Ya habíamos visto que con $F = 1$, $E = x + 2$ esto anda.

Pero si hago $F = 0$ y $E = x + 1$, también anda!

Y si hago $F = 137$ y $E = x + 138$, también anda.

Si hago $F = x + y + 3$ y $E = 2 * x + y + 4$, también anda.

Hay infinitas soluciones para E y F .

Veamos la wp:

```
wp.(x, y := x + F, E).(y = x + 1)  
≡ { def. := }  
  (y = x + 1)(x ← x + F, y ← E)  
≡ { sust. }  
  E = (x + F) + 1  
≡ { elijo F = 0 porque se me canta y porque es lo más simple que se me ocurre }  
  E = (x + 0) + 1  
≡ { elijo E = x + 1 }  
  True
```

Con esta opción el programa queda:

{ True }

$x, y := x + 0, x + 1$

{ $y = x + 1$ }

o más simple:

{ True }

$y := x + 1$

{ $y = x + 1$ }

Otra opción:

$\equiv \{ \text{elijo } \underline{\mathbf{E = 0}} \text{ porque se me canta y porque es lo más simple que se me ocurre} \}$

$0 = (x + F) + 1$

$\equiv \{ \text{arit.} \}$

$F = -x - 1$

$\equiv \{ \text{elijo } \underline{\mathbf{F = -x - 1}} \}$

True

Con esta opción el programa queda:

{ True }

$x, y := x + (-x - 1), 0$

{ $y = x + 1$ }

o más simple:

{ True }

$x, y := -1, 0$

{ $y = x + 1$ }

Práctico 6 - Ejercicio 3c

Encontrar E tal que valga la terna:

```
Const A, B : Nat;  
Var q, r : Nat;  
  { A = q * B + r }  
  q := E ;  
  r := r - B  
  { A = q * B + r }
```

¿Sale a ojo? Parece que $E = q + 1$.

¿Sale derivando? Supongamos la premisa $(A = q * B + r)$ y veamos la wp:

```
wp.(q := E ; r := r - B).(A = q * B + r)  
≡ { def wp para ; }  
  wp.(q := E).( wp.(r := r - B).(A = q * B + r) )  
≡ { def wp para := y sustitución }  
  wp.(q := E).( A = q * B + (r - B) )  
≡ { def wp para := y sustitución }  
  A = E * B + (r - B)  
≡ { reemplazo A por hip. }  
  q * B + r = E * B + r - B  
≡ { arit. para sacar r }  
  q * B = E * B - B  
≡ { sumamos B }
```

$q * B + B = E * B$
 $\equiv \{ \text{factor común} \} \ // \text{ podemos dividir B a ambos lados? } \mathbf{NO}$ porque no sabemos si $B \neq 0$ o no.
 $(q + 1) * B = E * B$
 $\equiv \{ \text{elijo } \mathbf{E = q + 1} \}$
 $(q + 1) * B = (q + 1) * B$
 $\equiv \{ \text{lógica} \}$
 True

Práctico 6 - Ejercicio 6d

¿Qué hace el programa? Este programa toma un arreglo A y un valor E,
 y si E está en el arreglo, calcula en i una posición de A a donde está el valor E.

¿Es correcto el programa? No porque siempre se pasa en una posición cuando encuentra el elemento.

¿Cómo lo arreglamos? Una solución es meter el incremento " $i := i + 1$ " como cuerpo de la 2da rama del if.

Otra solución: poner un " $i := i - 1$ " al final. ¿Anda? Sí, pero queda feo.

Terminación de ciclos

Retomamos el programa que vimos la clase pasada:

```
Var i,r : Int;  
{ P: i = A ∧ r = B ∧ i ≥ 0 }  
do i ≠ 0 →  
    i, r := i - 1, r + 1  
od  
{ Q: r = A + B }
```

Observación: Este programa es equivalente a uno mucho más simple. ¿cuál?

```
{ P: i = A ∧ r = B ∧ i ≥ 0 }  
i, r := 0, r + i  
{ Q: r = A + B }
```

Habíamos dicho que para demostrar la terna del ciclo debemos dar un predicado INV (el invariante) tal que valen las siguientes cosas:

1. La precondition implica al invariante:

$$P \Rightarrow INV.$$

Otra forma de decirlo: el invariante vale al comenzar el ciclo.

2. El invariante se preserva al ejecutar el cuerpo del ciclo:

$$\{ INV \wedge i \neq 0 \} \ i, r := i - 1, r + 1 \ \{ INV \}$$

3. Al terminar el ciclo, el invariante implica la postcondición:

$$INV \wedge \neg (i \neq 0) \Rightarrow Q$$

Estas tres condiciones no son suficientes para demostrar un ciclo.

Falta una cuarta condición:

4. El ciclo termina.

Existe una función $t : \text{Estado} \rightarrow \text{Int}$ ("**función de cota**") tal que cumple las siguientes cosas:

a. Si estoy en el ciclo, entonces la cota es mayor o igual que 0:

$$(\text{INV} \wedge B) \Rightarrow t \geq 0$$

Equivalentemente, **si la cota es menor que cero, entonces el ciclo termina:**

$$(\text{INV} \wedge t < 0) \Rightarrow \neg B \quad (\text{la guarda se hizo falsa})$$

b. La **cota se va achicando cada vez que se ejecuta el cuerpo del ciclo:**

$\{ \text{INV} \wedge B \wedge t = T \}$ // fijo el valor anterior de la cota en T (T es una const. de especificación)

$i, r := i - 1, r + 1$ // este es el cuerpo del ciclo

$\{ t < T \}$ // al terminar de ejecutar, la cota vale menos de lo que valía antes.

Tratemos de sacar a ojo alguna función de cota para nuestro programa:

```
Var i, r : Int;  
{ P: i = A  $\wedge$  r = B  $\wedge$  i  $\geq$  0 }  
do i  $\neq$  0  $\rightarrow$   
  i, r := i - 1, r + 1  
od  
{ Q: r = A + B }
```

Propuesta: $t = i$.

¿Decrece con el cuerpo del ciclo? Sí, se achica en 1.

¿Cuando estoy en el ciclo (o sea vale $\text{INV} \wedge B$), es $t \geq 0$? Sí, ya que INV dice $i \geq 0$.

¿Cuando $i < 0$, el ciclo termina? No puede suceder que $i < 0$ ya que el invariante me dice $i \geq 0$.

Encontrar la cota requiere de cierta creatividad.

Para dar las cotas vamos a mirar primero las guardas de los ciclos.

Ejemplo: Si mi guarda es “algo variable” < “algo constante” ($i < N$), la cota puede ser

$t = \text{“algo constante”} - \text{“algo variable”}$ ($t = N - i$)